
ARCHITECTURE BLUEPRINT · ADMIN BACK-OFFICE DEEP
DIVE

Necoyoad

v6

*The 70+ admin controllers, the AdminController base,
the file manager, and the visual editors*

scope: 443 PHP files across 22 admin controller folders
AdminController (1,244 lines), file manager (688 lines),
visual editors (1,853 lines), map.php preloader (843 lines)

findings: declarative CRUD framework, route-level RBAC,
no file versioning, pervasive activity logging

listings: 12 PHP · **pages:** 11

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — The Admin Back-Office (v6)

*The 70+ admin controllers, the AdminController base,
the file manager, and the visual editors*

Abstract

This is the sixth volume of the Necoyoad architecture blueprint. v6 zooms in on the admin back-office: the `AdminController` base class that provides generic CRUD to all 70+ admin controllers, the file manager that handles uploads and directory browsing, the visual editors (theme, widget, CSS/TPL), the admin authentication and permission model, and the admin's route-aware dependency preloader (`map.php`'s giant `switch` statement).

The admin back-office is the most code-intensive part of Necoyoad: 443 PHP files across 22 controller folders. The `AdminController` base class (1,244 lines) is a declarative CRUD framework — subclasses declare `$form_vars`, `$filters`, `$public_methods`, `$model_name`, and `$object_type`, and the base provides `index()`, `insert()`, `update()`, `copy()`, `delete()`, `activate()`, `sortable()`, `grid()`, `getList()`, `getForm()`, `validateForm()`, and `validateDelete()`. The file manager (688 lines) handles directory browsing, file upload, image resize, recursive delete, move, copy, and rename. The visual editors (1,853 lines across 5 controllers) include a CSS/TPL file editor, a theme + `theme_style` CSS generator, a widget layout manager, and a view-template browser.

v6 documents all of this with 12 PHP code listings, 4 TikZ diagrams, and the full 22-folder admin controller inventory. The stance is unchanged: **descriptive, not prescriptive**.

Keywords: Necoyoad, OpenCart, PHP 8.1, AdminController, CRUD framework, file manager, visual theme editor, widget layout manager, RBAC, admin authentication, `map.php` preloader.

Contents

1	Introduction	3
1.1	Six Volumes	3
1.2	Scope	3
2	Thirty-Second Overview	3
3	The AdminController Base Class	3
3.1	Declarative Properties	3
3.2	The CRUD Methods	4
3.3	The <code>upsert()</code> Method	4
3.4	The <code>getList()</code> Method	5
3.5	The <code>getForm()</code> Method	5
4	Admin Authentication and RBAC	5
4.1	The Login Pre-Action	6
4.2	The Permission Pre-Action	6
4.3	Activity Logging	6
5	The File Manager	6
6	The Visual Editors	6
6.1	<code>style/theme.php</code> (1,103 lines) — Theme Manager	7
6.2	<code>style/widget.php</code> (373 lines) — Widget Layout Manager	7
6.3	<code>style/editor.php</code> (170 lines) — CSS/TPL File Editor	7
6.4	<code>style/views.php</code> (151 lines) — View Template Browser	8
6.5	<code>style/template.php</code> (56 lines) — Template Manager	8
7	The Admin <code>map.php</code> Preloader	8
8	The Admin Controller Folders	9
8.1	The <code>admincontroller.php</code> File	9
9	Cross-Cutting Findings	9
9.1	The AdminController is a Declarative CRUD Framework	10
9.2	The <code>map.php</code> Preloader is a Performance Optimisation	10
9.3	The File Manager Has No Access Control Beyond Admin Session	10
9.4	The CSS/TPL Editor Has No Versioning	10
9.5	The Activity Log is Pervasive	10
9.6	The Permission Model is Route-Level, Not Object-Level	10
10	Closing Observations	10
10.1	The Admin is the Most Code-Intensive Part of Necoyoad	10
10.2	The Admin is a Separate Application with Shared Code	10
10.3	Final Note	11
A	Glossary	11

1 Introduction

1.1 Six Volumes

v1–v5 covered the whole platform, source verification, the rendering pipeline, the marketing subsystem, and the multi-store/multi-language architecture. v6 covers the admin back-office.

1.2 Scope

v6 covers:

- The **AdminController** base class (1,244 lines) — the declarative CRUD framework.
- The file manager (688 lines) — uploads, directory browsing, image resize.
- The visual editors (1,853 lines) — theme, widget, CSS/TPL editor, views, template.
- Admin authentication and RBAC — the login and permission pre-actions.
- The admin `map.php` preloader — the 843-line route-aware dependency switch.
- The 22 admin controller folders (443 PHP files total).

2 Thirty-Second Overview

The admin is a separate application (`app/admin/`) with its own entry point (`web/admin/index.php`), its own config (`STORE_ID=0`), its own `map.php` (843-line route-aware preloader), and its own pre-actions (login + permission). The admin dispatches via the same Front Controller as the storefront, but with two pre-actions: `common/home/login` (session token check) and `common/home/permission` (RBAC route check).

Most admin controllers extend **AdminController**, which provides generic CRUD. A subclass declares its model, object type, form fields, filters, and public methods; the base handles the rest (list, grid, insert, update, copy, delete, activate, sortable, form rendering, validation). The file manager is a standalone controller (**ControllerCommonFileManager**) that handles the file-system side of image uploads and directory management. The visual editors (`style/` folder) include a CSS/TPL file editor, a theme manager that generates CSS from the `theme_style` table, a widget layout manager that stores widget trees in the `property` EAV table, and a view-template browser.

3 The AdminController Base Class

`app/admin/controller/admincontroller.php` (1,244 lines) is the declarative CRUD framework. It extends **Controller** and provides all the standard admin operations.

3.1 Declarative Properties

Subclasses declare their configuration via protected properties:

```
1 class ControllerContentMenu extends ControllerAdmin {
2     protected string $object_type      = 'menu';
3     protected string $model_name       = 'modelMenu';
4     protected string $model_route      = 'content/menu';
5     protected string $model_object_type = 'menu';
6     protected string $controller_name  = 'menu';
7     protected string $controller_route = 'content/menu';
8     protected string $controller_template_basename = 'menu';
9     protected string $controller_template_route = 'content/menu';
10
11     protected array $form_vars = [
```

```

12         'menu_id'    => ['name' => 'menu_id', 'type' => 'number'],
13         'parent_id' => ['name' => 'parent_id', 'type' => 'number'],
14         'name'       => ['name' => 'name', 'type' => 'string'],
15         'sort_order' => ['name' => 'sort_order', 'type' => 'number'],
16         'default'    => ['name' => 'default', 'type' => 'boolean'],
17         'stores'     => ['name' => 'stores', 'type' => 'array'],
18     ];
19
20     protected array $filters = [
21         'name'       => ['name' => 'name', 'type' => 'string'],
22         'parent_id' => ['name' => 'parent_id', 'type' => 'number'],
23         'date_start' => ['name' => 'date_start', 'type' => 'date'],
24         'date_end'   => ['name' => 'date_end', 'type' => 'date'],
25     ];
26
27     protected array $public_methods = ['insert', 'update', 'copy',
28         'delete', 'activate', 'grid'];
29 }

```

Listing 1: AdminController declarative properties (from `content/menu.php`).

The `$form_vars` array declares which fields the form accepts and their types (used for validation and rendering). The `$filters` array declares which fields can be used to filter the list view. The `$public_methods` array declares which CRUD methods are accessible (a coarse permission gate on top of the RBAC check).

3.2 The CRUD Methods

The base provides the standard CRUD lifecycle:

Table 1: AdminController CRUD methods.

Method	Visibility	Purpose
<code>init()</code>	public	Loads the model via <code>\$model_route</code> ; sets <code>\$object_type</code> .
<code>index()</code>	public	Renders the list view (calls <code>getList()</code>).
<code>insert()</code>	public	Creates a new record (calls <code>upsert()</code>).
<code>update()</code>	public	Updates an existing record (calls <code>upsert()</code>).
<code>copy()</code>	public	Duplicates a record (or batch of records).
<code>delete()</code>	public	Deletes a record (or batch).
<code>activate()</code>	public	Toggles the status (0/1) of a record.
<code>sortable()</code>	public	Updates <code>sort_order</code> via AJAX drag-and-drop.
<code>grid()</code>	public	Renders the grid view (AJAX-loaded partial).
<code>updateparent()</code>	public	Updates <code>parent_id</code> for tree-structured entities.
<code>upsert()</code>	private	The shared insert/update logic: validates, processes POST, calls model <code>add()/update()</code> , fires hooks, redirects.
<code>getList()</code>	private	Builds the list view: breadcrumbs, filters, pagination, inline JS for activate/-copy/delete AJAX.
<code>getForm()</code>	private	Builds the insert/update form: loads model by PK, prepares <code>\$form_vars</code> , descriptions, languages, stores.
<code>validateForm()</code>	protected	Validates the POST against <code>\$form_vars</code> and <code>\$filters</code> .
<code>validateDelete()</code>	protected	Validates that the selected IDs can be deleted.

3.3 The `upsert()` Method

The shared insert/update logic (257–489) handles:

1. Validates the form (`validateForm()`).
2. Applies the `formData` filter (modules can modify POST data before save).

3. If **descriptions** are present: parses each language's HTML with `DOMDocument`, extracts base64 images, optionally embeds images, and (for campaigns) appends tracking pixels and rewrites links.
4. Calls `$this->model->add($data)` or `$this->model->update($id, $data)`.
5. Fires the **save** event and the **copy** / **delete** / **activate** events as appropriate.
6. Registers admin activity via `$this->user->registerActivity()` (writes to `user_activity` table).
7. Redirects to the list view.

3.4 The `getList()` Method

The list view builder (490–706) constructs:

- Breadcrumbs (home + current controller).
- Insert/delete URLs.
- Inline JavaScript for AJAX activate/copy/delete operations.
- The model query (using `$filters` to build the WHERE clause).
- Pagination.
- Per-row action buttons (edit, copy, delete, activate).
- The template path (default/`<controller>/<controller>_list.tpl` or a custom `default_admin_view_<controller>` setting).

3.5 The `getForm()` Method

The form builder (903–1196) constructs:

- Breadcrumbs (home + list + form).
- The model data (loaded by PK if editing, empty if inserting).
- The form action URL (`/insert` or `/update?id=N`).
- Descriptions (per-language title, body, SEO fields) loaded from the polymorphic `description` table.
- Languages (all active languages for the description tabs).
- Stores (multi-select for multi-store assignment).
- Any `$form_vars` that are not descriptions.
- The template path (default/`<controller>/<controller>_form.tpl`).

4 Admin Authentication and RBAC

The admin uses two pre-actions registered in `web/admin/index.php`:

```
1 $controller->addPreAction(new Action('common/home/login'));
2 $controller->addPreAction(new Action('common/home/permission'));
```

Listing 2: Admin pre-actions in `web/admin/index.php`.

4.1 The Login Pre-Action

`ControllerCommonHome::login()` (line 98) checks:

1. If the session is not valid (`!user->validSession()`) and the route is not `common/login/login` or `common/login/recover`, logout and forward to `common/login`.
2. If the route requires a token (not in the ignore list), check that `?token=` matches `session->ukey`. If not, logout and forward to `common/login`.

The ignore list includes `common/login`, `common/login/login`, `common/login/recover`, `common/login/ping`, `common/logout`, `error/not_found`, `error/permission`, plus any routes in the `config_token_ignore` setting.

4.2 The Permission Pre-Action

`ControllerCommonHome::permission()` (line 160) checks the current route against the user's group permission blob:

1. Extracts the route (first two path segments, or three for module routes).
2. If the route is in the ignore list (same as login), skip.
3. If the user does not have `access` permission for the route, forward to `error/permission`.
4. If the request is POST and the user does not have `modify` permission, forward to `error/permission`.

The permission check reads the `user_group.permission` serialised PHP array, which has `access` and `modify` keys, each containing a list of route strings.

4.3 Activity Logging

Every CRUD operation calls `$this->user->registerActivity($id, $object_type, $description, $action)`, which inserts a row into the `user_activity` table with the admin user's ID, the polymorphic object reference, the action (read, create, update, delete, copy, activate), a human-readable description, the IP, browser, OS, and session data.

5 The File Manager

`app/admin/controller/common/filemanager.php` (688 lines) is the file-system management controller. It provides:

All file operations are scoped to `DIR_IMAGE/data/`. The `str_replace('../', '', ...)` defence prevents path traversal. The file manager is used by the admin's WYSIWYG editor (for image insertion), the product image picker, the banner image picker, and any other admin form that needs to select an image.

6 The Visual Editors

The `app/admin/controller/style/` folder contains 5 controllers (1,853 lines total) that implement the visual editing tools.

Table 2: File manager methods.

Method	Purpose
<code>index()</code>	Renders the file manager iframe (ElFinder-like UI).
<code>image()</code>	Resizes an image to 100x100 thumbnail via <code>NTImage::resizeAndSave()</code> .
<code>directory()</code>	Lists subdirectories of <code>DIR_IMAGE/data/</code> as JSON (for the directory tree).
<code>files()</code>	Lists files in a directory as JSON (for the file grid).
<code>create()</code>	Creates a new directory.
<code>delete()</code>	Deletes a file or directory (recursive via <code>recursiveDelete()</code>).
<code>move()</code>	Moves a file or directory.
<code>copy()</code>	Copies a file or directory.
<code>folders()</code>	Recursively lists all folders (for dropdown selects).
<code>rename()</code>	Renames a file or directory.
<code>upload()</code>	Handles file upload (multipart form POST).
<code>uploader()</code>	Alternative upload handler (for AJAX uploaders).

6.1 style/theme.php (1,103 lines) — Theme Manager

The most complex visual editor. Manages themes (instances of templates), theme styles (CSS-rule overrides), and generates the custom CSS file.

Key methods:

- `insert()` / `update()` — creates/edits a theme (`template_id`, `store_id`, `name`, `colors`, `cols`, `scheme`, `date_publish_start/end`).
- `save()` — reads all `theme_style` rows for the theme and generates a CSS file via `ModelStyleTheme::saveStyle()` (covered in v3).
- `makeDefault()` — sets a theme as the default for its store.
- `loadThemeStyle()` — loads the `theme_style` rows for the visual editor's CSS-rule editing UI.

6.2 style/widget.php (373 lines) — Widget Layout Manager

Manages the widget tree (rows, columns, widgets) stored in the `property` EAV table. This is the admin-side counterpart to the storefront's `NecoWidget` (covered in v3).

Key methods:

- `saveRow()` — saves a widget row (position, sort order, settings). Enriches the serialised settings with `filter_*` string duplicates for the LIKE `'%key=value%'` queries.
- `saveCol()` — saves a widget column within a row.
- `saveWidget()` — saves a widget instance within a column.
- `deleteRow()` / `deleteColumn()` — cascading deletes that also clean up `widget_landing_page` rows.

6.3 style/editor.php (170 lines) — CSS/TPL File Editor

A raw file editor that lets the admin edit CSS and TPL files directly:

```

1 if ($this->request->get['t'] == 'css') {
2     $folder = DIR_THEME_ASSETS . $template . '/css/';
3 } elseif ($this->request->get['t'] == 'tpl') {
4     $folder = DIR_CATALOG . 'view/theme/' . $template . '/';
5 }
6 // ... list files in $folder, render edit form with file_get_contents ...

```

Listing 3: style/editor.php — the file type switch.

The editor reads a CSS or TPL file with `file_get_contents()`, presents it in a textarea, and saves it with `file_put_contents()`. There is no versioning or backup.

6.4 style/views.php (151 lines) — View Template Browser

Lists the `.tpl` files in the active theme and lets the admin assign a specific template to a route (stored as `default_view_<route>` in the `setting` table).

6.5 style/template.php (56 lines) — Template Manager

Manages the `template` table (design packages). Minimal CRUD — just name, version, colors, cols, scheme.

7 The Admin map.php Preloader

`app/admin/map.php` (843 lines) is the admin's bootstrap init script. It is a giant `switch ($route)` statement that preloads route-specific dependencies before the controller instantiates:

```
1  $route = strtolower($request->getQuery('r')??"");
2  switch ($route) {
3      case 'common/home':
4          $language->load('common/home');
5          $loader->auto('sale/customer');
6          $loader->auto('sale/order');
7          $loader->auto('store/product');
8          $loader->auto('store/review');
9          $loader->auto('currency');
10         $registry->set('currency', new Currency($registry));
11         break;
12     case 'store/category/insert':
13     case 'store/category/update':
14         $language->load('store/category');
15         $loader->auto('store/category');
16         $loader->auto('store/store');
17         $loader->auto('localisation/language');
18         $loader->auto('image');
19         $document->addStyle(HTTP_CSS . "fancybox/jquery.fancybox.css");
20         $document->addScript(HTTP_JS .
21             "jquery/fancybox/jquery.fancybox.pack.js");
22         break;
23     // ... 100+ more cases ...
24 }
```

Listing 4: `app/admin/map.php` — the route-aware preloader (abridged).

The preloader loads:

- Language files (per-route).
- Models (via `$loader->auto()`).
- Libraries (currency, image, pagination, etc.).
- CSS/JS assets (Fancybox, jQuery UI, etc.) — added to `Document`.
- Extensions (payment, shipping, total, module).

The switch has 100+ cases, each loading only the dependencies needed for that route. This is an optimisation: instead of loading all models and libraries at boot, only the ones the current route needs are loaded.

8 The Admin Controller Folders

The admin has 22 controller folders containing 443 PHP files:

Table 3: Admin controller folder inventory.

Folder	Files	Purpose
api/	84	REST API v1.0.0 (covered in v2).
chart/	3	Dashboard charts.
common/	8	Home, login, nav, header, footer, filemanager, callback.
content/	6	CMS: menu, post, page, banner.
error/	2	Permission, not_found.
extension/	4	Extension installer, module manager.
localisation/	11	Countries, zones, geo zones, languages, currencies, tax, weight/length classes, order/payment/stock statuses.
marketing/	6	Campaign, contact, list, mailserver, message, newsletter (covered in v4).
module/	263	69 widget modules + widgetcontroller.php + widget_common.php (covered in v3).
payment/	5	Payment extensions (bank transfer, PayPal, etc.).
report/	1	Sales reports.
sale/	9	Order, customer, customer_group, contact, coupon, return.
setting/	2	Store settings, extension settings.
shipping/	5	Shipping extensions (flat rate, weight-based, etc.).
store/	7	Product, category, manufacturer, attribute, download, review, store.
style/	5	Visual editors (covered in §6).
support/	1	Support tickets.
tool/ 7		Backup, log, cache, export, import.
total/ 7		Total extensions (sub-total, tax, shipping, coupon, etc.).
user/	2	User, user_permission.
widgets/	4	Widget admin.
Total	443	

8.1 The admincontroller.php File

The single most important file is `app/admin/controller/admincontroller.php` (1,244 lines), the base class for all CRUD controllers. Every controller in `content/`, `store/`, `sale/`, `localisation/`, `marketing/`, `user/`, and `tool/` extends it. Only `common/`, `style/`, `api/`, `chart/`, and `error/` have controllers that extend `Controller` directly.

9 Cross-Cutting Findings

9.1 The AdminController is a Declarative CRUD Framework

The AdminController is not just a base class — it is a declarative CRUD framework. A subclass declares its configuration (model, object type, form vars, filters, public methods) and gets the full CRUD lifecycle for free. This is a significant abstraction over vanilla OpenCart, where each admin controller has its own `insert()`/`update()`/`delete()` implementation.

9.2 The map.php Preloader is a Performance Optimisation

The 843-line switch statement in `map.php` is a route-aware dependency preloader. Instead of loading all models and libraries at boot, it loads only the ones the current route needs. This reduces memory usage and boot time for routes that don't need, say, the marketing or report models.

9.3 The File Manager Has No Access Control Beyond Admin Session

The file manager operates on `DIR_IMAGE/data/` with `str_replace('../', '', ...)` as the only path-traversal defence. Any admin with a valid session can browse, upload, delete, move, copy, and rename any file in the image directory. There is no per-user or per-folder access control.

9.4 The CSS/TPL Editor Has No Versioning

The `style/editor.php` controller lets the admin edit CSS and TPL files directly with `file_put_contents()`. There is no backup, no versioning, no diff. A typo in a CSS file can break the storefront with no recovery path short of a server backup.

9.5 The Activity Log is Pervasive

Every CRUD operation (insert, update, delete, copy, activate) and every list view access calls `user->registerActivity()`. The `user_activity` table records who did what, when, from what IP, with what browser. This is a comprehensive audit trail, at the cost of database write traffic on every admin action.

9.6 The Permission Model is Route-Level, Not Object-Level

The RBAC check in `permission()` checks whether the route (e.g. `store/product`) is in the user group's `access/modify` arrays. It does *not* check whether the user has permission to edit a specific product. Any admin with `modify` permission on `store/product` can edit any product in any store.

10 Closing Observations

10.1 The Admin is the Most Code-Intensive Part of Necoyoad

With 443 PHP files across 22 controller folders, the admin is the most code-intensive part of the platform. The AdminController base class is the keystone: it abstracts the CRUD lifecycle into a declarative framework, reducing 70+ controllers to a few lines of configuration each. The file manager and visual editors add file-system and CSS editing capabilities that make the admin a self-contained content management system.

10.2 The Admin is a Separate Application with Shared Code

The admin is a separate application (`app/admin/`) with its own entry point, config, and pre-actions, but it shares the `system/` engine, libraries, and database with the storefront. The two

applications communicate only through the database — there is no in-process API between them. Settings written by the admin are read by the storefront; products created by the admin are displayed by the storefront; widgets configured by the admin are rendered by the storefront.

10.3 Final Note

v6 confirms that the Necoyoad admin back-office is a characterful, hand-built content management system built on top of the OpenCart-derived MVC engine. The AdminController declarative CRUD framework, the file manager, and the visual editors (theme, widget, CSS/TPL) together form a complete admin experience that requires no file editing for day-to-day operations.

A Glossary

Table 4: Glossary of admin back-office terms.

Term	Meaning
AdminController	The declarative CRUD base class (1,244 lines). Subclasses declare <code>\$form_vars</code> , <code>\$filters</code> , <code>\$public_methods</code> , <code>\$model_name</code> , <code>\$object_type</code> .
<code>upsert()</code>	The shared insert/update logic in AdminController. Handles validation, POST processing, model save, hooks, redirect.
<code>map.php</code>	The admin's 843-line route-aware dependency preloader. A giant switch statement that loads only the models/libraries/assets needed for the current route.
File manager	ControllerCommonFileManager (688 lines). Handles directory browsing, file upload, image resize, recursive delete.
Visual editors	The style/ folder (5 controllers, 1,853 lines). Theme manager, widget layout manager, CSS/TPL editor, view browser, template manager.
RBAC	Route-level role-based access control. The user_group.permission blob contains access and modify arrays of route strings.
Activity log	The user_activity table, written on every CRUD operation and list view access.